The UK Entry Route

Job-market research and a 10-project portfolio plan for
Data Analyst and AI Engineer / AI Solutions roles

Prepared 12 August 2026

Evidence base: 2,436 unique UK permanent job advertisements
(1,671 Data Analyst · 765 AI Engineer), six months to 11 August 2026
plus individual employer listings and careers-service guidance

Part 1 — What the UK market actually asks for

1. How this research was done, and what it can and cannot tell you

You asked for 100+ listings. I did something stronger and something weaker, and you should know which is which before you act on any of it.

Stronger: instead of hand-reading a hundred adverts, I used IT Jobs Watch, which parses the full population of UK permanent IT vacancies and publishes co-occurrence counts. The Data Analyst figures below are drawn from **1,671 unique permanent UK adverts** and the AI Engineer figures from **765**, both covering the six months to 11 August 2026. That is a far larger and less cherry-picked sample than 100 manually chosen listings, and every percentage is a real count, not an impression.

Weaker: I did not read 100 full job descriptions myself, so the qualitative texture — the exact phrasing of responsibilities, the tone of seniority expectations — comes from a small number of individual adverts plus the skill-frequency data. Where I infer rather than measure, I say so. I have not fabricated a single number.

> **One structural caveat that matters.** IT Jobs Watch indexes adverts that carry the job title in question. "Data Analyst" adverts therefore skew towards roles that call themselves that, and exclude a large volume of Reporting Analyst, MI Analyst and Insight Analyst roles that sit outside the digital/IT taxonomy — particularly in the NHS, local government, insurance and retail back offices. The true entry-level analyst market is *larger* than the numbers here suggest. The AI Engineer numbers, by contrast, are close to complete, because almost every such role is indexed as IT. Read the Data Analyst volume as a floor and the AI Engineer volume as roughly the whole thing.

2. Track A — the Data Analyst market

Volume, salary and direction of travel

Metric	6 months to 11 Aug 2026	Same period 2025	Same period 2024
Unique permanent UK adverts	**1,671**	454	835
Share of all permanent UK IT jobs	1.57%	0.91%	1.00%
Demand rank (of ~780 titles)	117	303	—
Median salary	£45,000	—	—
10th / 25th / 75th percentile	£31,250 / £36,250 / £62,500	£28,750 / £37,238 / —	£30,000 / £36,250 / —

The headline: Data Analyst demand has roughly **tripled year on year** and the title climbed 186 places in the demand rankings. This is not a market in retreat, whatever the general graduate-hiring gloom suggests. The 10th percentile at £31,250 tells you where genuinely junior roles sit, and the gap up to the £45,000 median tells you the advertised population is mostly mid-level — you are competing at the bottom of a wide distribution.

What Data Analyst employers ask for (co-occurrence across 1,671 adverts)

Skill	Ads	%	Skill	Ads	%
SQL	1,055	**63.14%**	Data Quality	176	10.53%
Business Intelligence	951	56.91%	Problem-Solving	142	8.50%
Power BI	929	**55.60%**	Actionable Insight	130	7.78%
Power Platform	929	55.60%	Data Analytics	121	7.24%
Python	915	**54.76%**	Visualisation	105	6.28%
Microsoft Excel	892	**53.38%**	Continuous Improvement	97	5.80%
Microsoft (general)	838	50.15%	Data Modelling	86	5.15%
Tableau	820	49.07%	Dashboard Development	79	4.73%
AI	791	**47.34%**	Data Governance	76	4.55%
Azure	763	**45.66%**	Stakeholder Management	72	4.31%
Azure Certification	719	43.03%	Stakeholder Engagement	67	4.01%
Azure AI	716	42.85%	Validation	64	3.83%
Data Analysis	297	17.77%	Data Visualisation	63	3.77%
Analytics	292	17.47%	Business Analysis	60	3.59%
Decision-Making	238	14.24%	Data Transformation	54	3.23%

Source: IT Jobs Watch, UK permanent, six months to 11 August 2026. Percentages are of the 1,671 adverts carrying "Data Analyst" in the title.

Three things in that table should change what you build.

First, Python is at 54.76% — essentially level with Excel. The old advice that analyst roles are Excel-and-SQL-only is out of date. Your Python is an asset here, not something to hide.

Second, AI appears in 47.34% of Data Analyst adverts. Nearly half. Employers are not hiring analysts and AI engineers into separate universes any more; they are asking analysts to be AI-literate. This is the single most important finding in this report for someone with your profile, because it means your two "tracks" are converging and a hybrid portfolio is not a hedge — it is the target.

Third, Azure sits at 45.66% with Azure AI at 42.85%. Note what is missing from the top 30 entirely: AWS. In UK analytics, the stack is Microsoft.

Where the jobs are

Location	Adverts (6 mo)	Median salary	YoY salary change
England	1,492	£45,000	−5.26%
UK excluding London	1,418	£45,000	+5.88%
North of England	483	£45,000	+5.88%
Midlands	263	£42,500	—
South East	257	£42,500	−8.11%
London	226	£56,125	−13.65%
Work from home	210	£46,000	−3.16%
North West	206	£45,000	+2.86%
— Manchester	72	£46,750	+10.00%
— Merseyside	49	£40,000	−23.81%
— Cheshire	19	£65,000	—
Scotland	82	£65,000	+49.20%

> **Liverpool / Manchester read.** Merseyside has 49 adverts in six months — roughly two a week — and its median fell 23.81%. Manchester has 72 adverts, a rising median, and is 35 minutes away by train. If you are Liverpool-based, **Manchester is your local market and Merseyside is a bonus**, and 210 explicitly remote adverts plus 1,418 outside London mean geography is not the binding constraint. Do not restrict searches to Liverpool; you will see a quarter of your real opportunity set.

Data Analyst skill classification

Tier	Skills	Evidence
Must have	SQL; Power BI (incl. DAX, Power Query); Excel; Python (pandas); stakeholder communication in writing	SQL 63%, Power BI 56%, Excel 53%, Python 55%
High value	Azure data services; a relational database you can speak to (PostgreSQL is fine); data modelling (star schema); data quality practice; AI literacy	Azure 46%, AI 47%, Data Quality 11%, Data Modelling 5%
Nice to have	Tableau; data governance; statistical testing; forecasting; dbt	Tableau 49% but largely interchangeable with Power BI in practice; governance 4.6%
Low value	Deep ML theory; React/Next.js; Django; Docker; AWS; Kubernetes	Absent from the top 30 co-occurrences entirely

Tableau at 49.07% deserves a note, because it looks like it demands equal investment. It does not. Very few adverts require both tools to a deep level; most say "Power BI or Tableau". Because Power BI edges it and because Power BI carries the Microsoft/Azure ecosystem that 50% of adverts also mention, **go deep on Power BI and learn just enough Tableau to talk about the difference.** One dashboard rebuilt in Tableau to prove transferability is sufficient.

3. Track B — the AI Engineer / AI Solutions market

Volume, salary and direction of travel

Metric	Value (6 months to 11 Aug 2026)
Unique permanent UK adverts, "AI Engineer"	**765** (0.72% of all permanent UK IT jobs)
Demand rank change year on year	**Up 351 places**, to rank 266
Live vacancies at time of research	1,304
Median salary	**£87,500** (+2.94% YoY)
10th / 25th / 75th / 90th percentile	£53,750 / £62,000 / £100,000 / £112,500
Adjacent: "AI Engineering"	544 adverts, median £93,750
Adjacent: "Gen AI Engineer"	31 adverts, median £85,000
Adjacent: "Agentic AI Engineer"	3 adverts, median £100,000
Comparator: "Data Engineer"	1,960 adverts, median £70,000

> **Read the salary distribution before you read the growth rate.** A 10th percentile of £53,750 means that even the cheapest tenth of AI Engineer roles pay well above a graduate salary. The title is growing explosively — up 351 rank places — but it is growing as a *senior* title. Employers are converting experienced software and ML engineers into AI engineers, not hiring graduates into it. The 3 adverts for "Agentic AI Engineer" against 1,960 for "Data Engineer" should also calibrate how much of the online noise about agents reflects UK hiring.

What AI Engineer employers ask for (co-occurrence across 765 adverts)

Skill	Ads	%	Skill	Ads	%
AI (general)	713	93.20%	MLOps	73	9.54%
Python	538	**70.33%**	Problem-Solving	68	8.89%
Azure	440	**57.52%**	CI/CD	61	7.97%
AI Engineering	383	50.07%	GCP	59	7.71%
Azure AI	367	47.97%	Anthropic Claude	58	7.58%
Azure Certification	342	44.71%	Observability	54	7.06%
Software Engineering	228	**29.80%**	Decision-Making	54	7.06%
LLM	213	**27.84%**	Full-Stack Development	52	6.80%
Machine Learning	147	19.22%	Microsoft	50	6.54%
Retrieval-Augmented Generation	136	**17.78%**	Degree	49	6.41%
Generative AI	120	15.69%	OpenAI	49	6.41%
AWS	86	11.24%	Security Cleared	48	6.27%
LangChain	86	11.24%	Data Science	45	5.88%
Prompt Engineering	82	10.72%	Mentoring	44	5.75%
Workflow	78	10.20%	Agile	43	5.62%

Source: IT Jobs Watch, UK permanent, six months to 11 August 2026.

The pattern here is blunt: **Azure at 57.52% against AWS at 11.24% and GCP at 7.71%.** UK AI engineering is a Microsoft market, driven by enterprises who already buy Microsoft 365 and route their AI spend through the same

procurement. Your AWS listing on your CV is close to worthless for this track in the UK; the same effort spent on Azure AI Foundry, Azure OpenAI and Azure AI Search would be five times more visible.

Note also that **Software Engineering (29.80%) outranks Machine Learning (19.22%)**. Employers are describing a software role that touches models, not a research role. And "Mentoring" at 5.75% is a quiet seniority tell — a meaningful slice of these adverts expect you to lead other people.

AI Engineer skill classification

Tier	Skills	Evidence
Must have	Python to a software-engineering standard; API design (FastAPI); LLM application patterns; Azure AI services; Git	Python 70%, Azure 58%, Software Eng 30%, LLM 28%
High value	RAG with real retrieval evaluation; CI/CD; Docker; observability and cost tracking; prompt engineering; structured output/schema validation	RAG 18%, MLOps 10%, CI/CD 8%, Observability 7%
Nice to have	LangChain; classical ML; AWS or GCP; full-stack; vector database internals	LangChain 11%, ML 19%, AWS 11%, Full-stack 7%
Low value	Model training from scratch; deep-learning research; fine-tuning as a headline skill; agent frameworks as a headline skill	Agentic AI Engineer: 3 adverts nationally

4. The overlap — where one unit of effort buys two tracks

Skill	DA demand	AI Eng demand	Cross-track leverage	Your current position
Python	54.76%	70.33%	**Highest** — the only skill in both top 5	Have it; needs engineering rigour
Azure ecosystem	45.66%	57.52%	**Highest** — and you currently have none of it	Gap. Biggest single gap in this report
AI / LLM literacy	47.34%	93.20%	**Very high** — the convergence point	Strong. Your best differentiator
SQL	63.14%	Implicit via data work	High for DA, moderate for AI	Have it; needs window functions/CTEs depth
Problem-solving / decision-making	14.24% + 8.50%	8.89% + 7.06%	High — demonstrated through writing	Unproven to a stranger. Fix via READMEs
Data quality / validation	10.53% + 3.83%	Via eval discipline	High — underrated bridge	Strong (dissertation work)
APIs / FastAPI	Low	Core	Low cross-over, high AI value	Have it
Power BI / DAX	55.60%	Negligible	DA only — but non-negotiable there	Gap
Docker / CI/CD	Negligible	7.97%+	AI only	Have Docker; CI/CD unproven
React / Next.js / TypeScript	Negligible	6.80%	Low — but useful for demos	Have it. Use it, do not sell it
AWS	Absent from top 30	11.24%	**Low in the UK**	Have some. Deprioritise
Django	Absent	Absent	None	Remove from CV headline

> **The overlap verdict.** Four things buy you both tracks at once: **Python, Azure, AI/LLM fluency, and the ability to write about data quality and decisions.** Everything else is track-specific. If you learn one new platform in the next three months, learn Azure — it appears in 46% of analyst adverts and 58% of AI engineer adverts, and you have zero exposure to it today.

5. Which track should be primary?

Criterion	Data Analyst	AI Engineer	Winner
Advert volume (6 mo)	1,671 (an undercount)	765	DA, by 2.2× or more
10th-percentile salary as a seniority proxy	£31,250 — genuine junior roles exist	£53,750 — almost none do	DA, decisively
Explicit graduate/trainee adverts observed	Multiple ("Data Analyst Trainee", £30k+)	Few; Sparta Global at £25–26k is typical	DA
Fit with your existing skills	Python, SQL, PostgreSQL, statistics — strong	FastAPI, RAG, LLM — strong but unproven commercially	Tie
Portfolio leverage (can a project substitute for a job?)	High — the work is legible in a dashboard	Moderate — employers discount unoperated demos	DA
North West availability	206 NW adverts; Manchester rising	156 across the whole North of England	DA
Growth trajectory	Tripled YoY, rank +186	Rank +351	AI Eng
Salary ceiling	£62,500 at 75th percentile	£100,000 at 75th percentile	AI Eng
Transition potential into the other track later	Strong — analyst→AI is a well-worn path, and 47% of analyst roles already touch AI	Weak in reverse — nobody moves AI engineer→analyst	DA

> **Primary track: Data Analyst, with a deliberate AI specialism. Secondary track: AI Engineer / AI Solutions.**
>
> The reasoning is not that AI engineering is a worse job. It is that the AI Engineer market has almost no junior door, and the Data Analyst market has one that is getting wider fast. With no internship and no industry experience, you need a market that hires on potential. Analytics does; AI engineering currently does not, outside a handful of consultancy graduate schemes.
>
> The strategic move is not to abandon AI. It is to **enter as the analyst who can build AI systems** — a position that 47.34% of analyst adverts now explicitly want and that almost no other junior candidate can occupy. You then convert to AI Engineer from inside a company in 18–30 months, at a salary the external market would never have offered you as a graduate.

6. The recruiter's thirty seconds

I have written this section as the hiring manager, not the careers adviser. It is unflattering in places.

What gets you shortlisted

- **Keyword match in the top third of page one.** Screening is often automated and always fast. If "SQL", "Power BI" and "Python" are not visible above the fold, the rest does not get read.
- **One live, clickable thing.** A published dashboard or a deployed API that loads in under five seconds. Not a GitHub repo — a working artefact. Most reviewers will not clone anything.
- **A number attached to an outcome.** "Identified £340k of annual revenue leakage across 1.2m transactions" survives a skim. "Performed exploratory data analysis" does not.
- **Evidence you have spoken to a non-technical person.** An executive summary written for a director is worth more than a third model.

What makes a project look like a university assignment

- A Jupyter notebook as the deliverable. This is the single strongest negative signal in a junior analytics portfolio.
- Titanic, Iris, the Kaggle house-prices set, or any dataset the reviewer has seen forty times.
- A "Conclusion" section that summarises what you did rather than what should now happen.
- Accuracy reported without a baseline, or reported to four decimal places.
- No stated business owner. If you cannot name whose job this changes, it is coursework.
- A README that starts with installation instructions rather than the problem.

What makes a project look commercially relevant

- It starts with a decision someone has to make, and ends with a recommendation and a cost.
- It handles the ugly parts — missing data, duplicate keys, changing definitions — and says so.
- It has a limitations section. Nothing signals seniority faster than a candidate who volunteers what their work cannot do.
- It is versioned, tested, and deployed somewhere with a URL.
- Somebody other than you could run it tomorrow.

What causes rejection

- **Ten repositories that are the same project.** Breadth is read as depth of confusion.
- **Unverifiable claims.** "Improved accuracy by 40%" with no baseline reads as inflation, and an interviewer will test it.
- **A CV listing thirty technologies.** With no experience to anchor them, a long list reads as tutorial completion, not capability. Yours currently lists Django, AWS, React, Next.js, TypeScript, Docker, LangChain and more — that breadth is actively hurting you for analyst roles.
- **Dead links.** An unreachable demo is worse than no demo, because it proves you did not check.
- **Applying to roles requiring 3–5 years with no covering note.** Fine to apply; not fine to apply silently.

7. "No industry experience → portfolio evidence" framework

What an employer cannot verify about you	The specific artefact that substitutes for it	Where it lives
You have analysed real business data	An end-to-end analysis on a messy public dataset, with a documented data-quality log listing every defect found and the decision taken	P1, P2
You can write production SQL	A versioned `/sql` directory with CTEs, window functions and a tested star-schema build, not queries pasted in a notebook	P2, P4
You can build BI that others use	A published Power BI report with a documented semantic model, measure catalogue, and row-level security	P1, P4
You can handle a stakeholder	A two-page executive brief with a recommendation, a cost, a risk and an ask — written for a named fictional role	P1, P3, P9
You understand business metrics	A KPI definition document: formula, grain, owner, refresh cadence, known caveats	P2, P3, P4
You have shipped software	A containerised FastAPI service with an OpenAPI spec, health checks and a public URL	P6, P7
You have deployed to cloud	An Azure deployment with infrastructure defined in code and a documented monthly cost	P6, P10
You have operated something	A monitoring dashboard showing latency, error rate and spend over time on your own live service	P6, P8
You have tested code others depend on	A test suite with a visible coverage badge and CI running on every push	P8, P10
You can evaluate an AI system	A gold-standard evaluation set you built by hand, with baseline comparison, failure taxonomy and per-query cost	P5, P8, P9
You know when not to use AI	A written trade-off note where you chose a rules engine or a classical model over an LLM, with the reasoning	P7
You have worked to a deadline with changing requirements	A repo whose issue tracker and commit history show scope changes handled over weeks, not one squashed commit	All
You can be trusted with sensitive data	A documented handling policy: what is redacted, what is logged, what never leaves the process	P10

8. Market gaps — where supply is thinnest

These are the places where employer demand and typical junior supply diverge most sharply. Every one of the ten projects is aimed at at least one of them.

Employers ask for	Juniors typically show	The gap you exploit
SQL in 63% of analyst adverts	pandas notebooks	A repository where SQL is the primary language and Python is the orchestrator, not the reverse
Power BI in 56%	matplotlib charts	A published, governed semantic model with documented DAX
Azure in 46–58%	local scripts, occasionally AWS free tier	Anything at all running on Azure. The bar here is astonishingly low
RAG in 18% of AI adverts	a chatbot over one PDF, no evaluation	A retrieval system with recall@k, faithfulness scoring and a failure taxonomy
Observability in 7%, MLOps in 10%	nothing — the demo runs once	A live service with a public metrics page showing uptime, p95 latency and cost
Data quality in 11%	silent `dropna()`	An explicit missingness diagnosis with justified handling and a sensitivity analysis
Stakeholder communication in 8%+	a technical README	Written executive briefs alongside the code
"Actionable insight" in 8%	descriptive statistics	Recommendations with an owner, a cost and a measurable success criterion

> **The biggest arbitrage in this report.** Observability and evaluation appear in roughly one in ten AI adverts and in approximately zero junior portfolios. A single project that shows a live AI service with a real cost-per-query chart and a hand-built evaluation set will differentiate you more than three additional chatbots. This is why P8 exists.

9. Skill priority matrix

Skill	DA demand	AI demand	Hiring importance	Your gap	Evidence needed	Priority
SQL (window fns, CTEs)	63.14%	Indirect	Critical	Moderate	Versioned SQL repo, tested models	**Tier 1**
Power BI + DAX + Power Query	55.60%	—	Critical (DA)	**Total**	2 published reports, semantic model doc	**Tier 1**
Python (engineering standard)	54.76%	70.33%	Critical	Small	Typed, tested, packaged code	**Tier 1**
Azure (Data + AI services)	45.66%	57.52%	Critical	**Total**	2 deployed workloads + AI-900/DP-900	**Tier 1**
Excel to a professional standard	53.38%	—	High	Unknown	Power Query model, not just formulas	**Tier 1**
Business writing / exec summary	8%+ explicit, 100% implicit	7%+	Critical	Unproven	One brief per project	**Tier 1**
LLM application patterns	47.34% (as "AI")	27.84%	High	Small	Two production-shaped systems	**Tier 2**
RAG + retrieval evaluation	—	17.78%	High	Moderate (eval side)	Gold set, recall@k, faithfulness	**Tier 2**
FastAPI / REST design	—	Core to 30% "software eng"	High (AI)	Small	OpenAPI spec, auth, versioning	**Tier 2**
Statistics & experiment design	Implicit	5.88%	High	Small — a genuine strength	Hypothesis tests with assumptions stated	**Tier 2**
Data modelling (star schema)	5.15% explicit, universal in practice	—	High	Moderate	A documented dimensional model	**Tier 2**
CI/CD + testing	—	7.97%	Moderate	Moderate	GitHub Actions on every repo	**Tier 3**
Docker	—	Implicit	Moderate	Small	Working compose file	**Tier 3**
Observability / cost tracking	—	7.06%	Moderate but rare	Large	Public metrics page	**Tier 3**
Tableau	49.07%	—	Moderate (substitutable)	Total	One rebuilt dashboard	**Tier 3**
Classical ML	—	19.22%	Moderate	Small	One honest end-to-end pipeline	**Tier 3**
LangChain	—	11.24%	Low	None	Mention, do not centre	**Tier 4**
AWS	Absent	11.24%	Low in the UK	Small	Keep on CV, stop investing	**Tier 4**
React / Next.js / TypeScript	Absent	6.80%	Low	None	Use for demo UIs only	**Tier 4**
Django	Absent	Absent	None	—	Remove from CV	**Tier 4**

Part 2 — The ten projects

10. How the ten were chosen

I generated a pool of 24 candidate projects from the demand data, then scored each against your weighting. Only the top ten survive. The full scoring for the survivors is below; the discarded pool is summarised after it, because knowing what you are *not* building is as useful as knowing what you are.

Scoring weights, as specified: employer demand 20, business value 15, technical depth 15, hiring signal 15, differentiation 10, interview value 10, production readiness 5, feasibility 5, role relevance 5.

#	Project	Track	Dem /20	Bus /15	Tech /15	Sig /15	Diff /10	Int /10	Prod /5	Feas /5	Rel /5	Total
P10	ImputeIQ — data quality & missingness platform	Hybrid	18	14	15	15	10	10	5	4	5	**96**
P1	NHS RTT waiting-list performance analytics	DA	20	15	11	14	8	9	4	5	5	**91**
P9	Governed natural-language analytics layer	Hybrid	18	13	14	14	10	10	4	3	5	**91**
P2	Revenue leakage & margin erosion (advanced SQL)	DA	19	14	12	14	8	9	4	5	5	**90**
P8	LLM evaluation & regression-testing harness	AI	15	12	14	15	10	10	5	4	4	**89**
P5	Grounded regulatory compliance assistant (RAG)	AI	17	13	13	13	8	10	4	4	5	**87**
P4	Automated MI reporting pipeline	DA	18	14	10	13	8	8	4	5	5	**85**
P6	Document intelligence extraction API	AI	16	13	13	12	7	9	5	4	5	**84**
P3	Churn & retention cohort intelligence	DA	17	13	11	12	6	9	3	5	5	**81**
P7	Service-desk triage automation with guardrails	AI	15	13	12	12	7	9	4	4	4	**80**

Why the scores fall where they do

- **P10 leads on differentiation and interview value, not on demand.** Nobody is advertising for a missingness-diagnosis specialist. It scores 96 because it is the only project in the set that a technical interviewer cannot exhaust in ten minutes, and because you have already built most of it — feasibility is 4/5 only because polishing it for a commercial audience is real work.
- **P1 scores maximum on demand** because NHS and public-sector analytics is the single largest employer of junior analysts in the North West, and the data is genuinely open.
- **P9 ties P1** because a governed text-to-SQL layer is the clearest possible demonstration that you belong in both tracks. Feasibility is 3/5 — it is the hardest thing here to do honestly.
- **P8 has the lowest demand score of the AI projects (15/20)** because "evaluation harness" is rarely a job title. It survives at 89 because it maps directly onto the biggest gap identified in section 8, and because it is the project most likely to make an interviewer stop and pay attention.
- **P3 and P7 score lowest** and are the first two to cut if time runs short. Churn analysis is well-trodden; triage automation is credible but crowded.

Cut from the pool, and why

Twitter/X sentiment dashboard (no business owner, saturated) · stock-price predictor (implies claims you cannot support) · movie recommender (no UK employer signal) · fine-tuned domain LLM (cost and time out of scope; near-zero UK junior demand) · multi-agent research swarm (3 UK adverts nationally) · computer-vision defect detection (off-track for both) · Kubernetes ML platform (Tier 4 skill) · Kaggle competition placement (signals hobby, not employment) · personal finance tracker (universally read as a tutorial) · blockchain anything (negative signal) · Kafka streaming pipeline (data engineering track, not yours) · standalone A/B testing framework (good, but subsumed by P3) · voice assistant (no demand) · CV-screening AI (legally fraught, and recruiters dislike being automated).

11. Portfolio allocation, and why it is not quite 4/4/2

You proposed four Data Analyst, four AI, two hybrid. The evidence supports that split numerically, so it stands — but with one important change of emphasis. Because AI appears in 47.34% of Data Analyst adverts, the two hybrid projects are not a hedge between the tracks; they are the most commercially aligned things in the portfolio. So: **same allocation, inverted emphasis.** The hybrids take the flagship slot and the second CV slot, not the leftover slots at the bottom of the page.

12. The projects

P1 — Referral-to-Treatment Performance Intelligence Data Analyst 91/100

Business problem
NHS trusts must report referral-to-treatment (RTT) waiting times and are penalised for 52-week breaches. Operational managers see breach counts after the fact, when nothing can be done. They cannot see which specialties are trending towards breach four weeks out, or whether a breach was driven by referral surge, capacity loss or administrative delay.

Target user
Directorate Operations Manager at an acute trust; secondary audience the ICB performance team.

Business process
Monthly RTT submission to NHS England, weekly performance meeting, escalation of at-risk pathways to specialty leads.

Pain point
Reporting is retrospective and pathway-level; the weekly meeting argues about whose numbers are right rather than about what to do.

Why this matters
Breach penalties and reputational risk are material, and elective recovery funding is tied to performance. This is the most common analytical job in the UK public sector.

Proposed solution
A Power BI report over a PostgreSQL star schema, with a four-week forward breach-risk indicator per specialty derived from waiting-list age distribution and clearance rate, plus a driver decomposition separating demand, capacity and administrative causes.

Expected KPI
Reduction in 52-week breaches through four weeks of warning on at-risk pathways; measured as the fall in "unforeseen" breaches from a current baseline of 100%.

Data sources
NHS England RTT waiting times statistics (open, monthly, provider and specialty level); Hospital Episode Statistics summaries; ONS population estimates for catchment normalisation. All genuinely public.

Technical architecture
Python ingestion → raw layer in PostgreSQL → SQL transformation to a star schema (fact_waiting_list, dim_provider, dim_specialty, dim_date) → Power BI import mode with a documented semantic model → published to Power BI Service.

Stack
Python (pandas, requests), PostgreSQL, SQL, Power BI, DAX, Power Query, Git.

AI/ML methodology
None, deliberately. A written note explains why a linear clearance-rate projection beats a model here: it is explainable to a clinician, it degrades predictably, and there are roughly 20 monthly observations per specialty.

Analytics methodology
Waiting-list age-band cohort analysis; clearance rate and its trend; a simple queueing projection; seasonal decomposition of referral volume; driver decomposition.

SQL requirements
Window functions for period-over-period and running clearance; CTEs for the pathway build; a slowly-changing dimension for provider reorganisations; a date spine.

Backend / API
None required — a scheduled Python ingestion job only.

Database
PostgreSQL with `raw`, `staging` and `marts` schemas.

Frontend
Power BI: an executive page, a specialty drill-down, a breach-risk watchlist, and a data-quality page showing suppression and revision rates.

Deployment
Published to Power BI Service with a shareable link; ingestion scheduled on GitHub Actions cron.

Testing
SQL assertion tests on the marts: uniqueness of grain, non-negative waits, referential integrity, row-count reconciliation against source totals.

Monitoring
Ingestion log table; a freshness indicator on the report face; an alert if source publication is late.

Security
Row-level security by provider, demonstrating that you understand not every user should see every trust.

Evaluation
Backtest the four-week breach-risk flag against the following month's actuals across the last 12 months; report precision and recall, and say so plainly if it is mediocre.

Scalability
Incremental refresh partitioned by month; note the point at which import mode should become DirectQuery.

GitHub structure
`/ingest`, `/sql/staging`, `/sql/marts`, `/tests`, `/powerbi` (pbip, source-controlled), `/docs/semantic-model.md`, `/docs/exec-brief.pdf`, `/docs/data-quality-log.md`.

README
Problem → who it is for → screenshot → live link → three findings → how the model is built → data-quality issues found → limitations → how to run.

Demo
Public Power BI link plus a three-minute walkthrough recorded as if presenting to the operations manager.

Build time / difficulty
2.5 weeks · Moderate

CV bullet
"Built an RTT performance report over a PostgreSQL star schema of provider-specialty-month records, introducing a four-week breach-risk indicator backtested at [X]% precision across 12 months, with row-level security by provider."

Interview questions
Why a star schema rather than one wide table? How does your grain survive a provider merger mid-year? What is the difference between a calculated column and a measure, and why does it matter here? Your breach flag has [X]% precision — would you ship that? What would a clinician object to?

Plausible UK users
Evidence: NHS England publishes this data precisely because trusts and ICBs are measured on it. *Reasonable inference:* acute NHS trusts, integrated care boards, independent-sector providers under NHS contract, health analytics consultancies.

P2 — Revenue Leakage and Margin Erosion Analysis Data Analyst 90/100

Business problem
A multi-channel retailer's gross margin is falling while revenue is flat. Finance sees the aggregate but cannot attribute it. Margin is leaking across discount depth, returns, delivery subsidy, price-file errors and supplier cost drift — and each sits with a different owner who blames the others.

Target user
Commercial Finance Manager; secondary audience the trading and category managers whose decisions are implicated.

Business process
Monthly margin review; annual supplier negotiation; weekly promotional sign-off.

Pain point
No single view attributes margin movement to cause, so promotional decisions are made on revenue uplift alone.

Why this matters
Margin leakage is typically the highest-value analysis a junior analyst can produce in retail, and it is the archetypal commercial-analyst interview scenario.

Proposed solution
A margin bridge decomposing period-over-period gross margin change into price, volume, mix, discount, returns and cost effects — delivered as SQL models plus a Power BI report, with a ranked list of leakage sources and a recommended action and owner for each.

Expected KPI
Gross margin percentage recovered; specifically, the top five leakage sources quantified in pounds per annum.

Data sources
UK Online Retail II (UCI, ~1m transactions, genuinely messy — cancellations, negative quantities, missing customer IDs); ONS retail sales index and CPI for deflation; a documented synthetic cost and promotion layer.

Synthetic data strategy
The source has no cost or promotion data. Rather than random numbers: generate costs from a category-level margin distribution calibrated to published UK retail sector margins; align promotional periods to the real UK retail calendar (Black Friday, Boxing Day, Easter); and inject a small rate of deliberate price-file errors. Seed the generator, commit it, and state clearly in the README which columns are synthetic.

Technical architecture
PostgreSQL raw → staging with a documented cleaning contract → fact_sales_line, dim_product, dim_customer, dim_date, dim_promotion → margin bridge model in SQL → Power BI.

Stack
PostgreSQL, advanced SQL, Python (generation and validation only), Power BI, DAX.

AI/ML methodology
None. Include a note explaining that attribution here is arithmetic, not prediction, and that reaching for a model would obscure the answer rather than sharpen it.

Analytics methodology
Price-volume-mix decomposition; RFM segmentation to test whether leakage concentrates in a customer segment; Pareto on leakage sources; basket-level contribution margin.

SQL requirements
This is the SQL showcase. Window functions (LAG, running totals, NTILE for deciles), a recursive CTE for the date spine, self-joins for cancellation matching, and a genuinely hard multi-step bridge query with each step materialised and commented.

Backend / Database / Frontend
No API. PostgreSQL. Power BI with a margin-bridge waterfall as the hero visual.

Deployment / Testing / Monitoring
Published Power BI report. The critical test: the sum of decomposed effects must equal total margin change to the penny, and a test asserts it. Refresh log.

Security / Scalability
Customer identifiers pseudonymised. Note the partitioning strategy at 100m rows.

Evaluation
The reconciliation test above, plus a sensitivity check showing how the conclusions shift under a different cost assumption.

GitHub structure

`/sql` as the primary directory with numbered, commented models; `/synthetic` with the seeded generator and its calibration note; `/tests`; `/docs/kpi-definitions.md`; `/docs/exec-brief.pdf`.

Build time / difficulty
2 weeks · Moderate

CV bullet
"Decomposed 12 months of gross-margin movement across 1m retail transactions into price, volume, mix, discount and returns effects using a reconciling SQL margin bridge, isolating five leakage sources worth £[X]k annually with an owner-level action for each."

Interview questions
Walk me through your margin bridge. Why does it reconcile exactly? What did you do with the 25% of rows missing a customer ID, and what did that cost you? Which of your five findings would you drop if the trading director pushed back? How would you productionise the refresh?

Plausible UK users
Reasonable inference: grocery and general retail, e-commerce, wholesale distribution, hospitality groups, consumer-goods manufacturers — all of which routinely advertise Commercial Analyst roles with exactly this remit.

P3 — Customer Churn and Retention Cohort Intelligence Data Analyst 81/100

Business problem
A subscription business knows its overall churn rate but not which cohorts churn, when in the lifecycle, or why. Retention spend is therefore untargeted — discounts go to customers who would have stayed anyway.

Target user
Head of Customer Retention; secondary audience the marketing team who own the intervention budget.

Business process
Monthly retention review; quarterly allocation of save-offer budget; win-back campaigns.

Pain point
Aggregate churn masks cohort dynamics; interventions land late and indiscriminately.

Why this matters
Retention is the second most common commercial analyst brief after margin, and it forces genuine statistical care about survivorship.

Proposed solution
Cohort retention curves by acquisition month and channel, survival analysis identifying hazard peaks in the lifecycle, driver analysis, and a costed intervention recommendation with an expected-value calculation.

Expected KPI
Retention at month 3 and month 12; cost per retained customer against the current blanket-discount baseline.

Data sources
Public telco or banking churn datasets, supplemented with a documented synthetic tenure and event history calibrated to published UK churn benchmarks.

Analytics methodology
Cohort retention matrices; Kaplan–Meier survival curves with confidence intervals; log-rank tests between segments; logistic regression for interpretation with coefficients reported as odds ratios; explicit treatment of survivorship bias and of why these associations are not causal.

AI/ML methodology
A gradient-boosted churn classifier as a secondary artefact, evaluated properly: a majority-class baseline, a temporal rather than random train/test split, precision-recall rather than accuracy (the classes are imbalanced), a calibration curve, and SHAP for attribution. The written comparison of the interpretable model against the accurate one is the actual point of the exercise.

SQL requirements
Cohort assignment with window functions; a gaps-and-islands query to reconstruct subscription spells from event data; the retention matrix pivot.

Stack / Database / Frontend
Python (pandas, lifelines, scikit-learn, SHAP), PostgreSQL, Power BI for the retention matrix and cohort heatmap.

Deployment / Testing / Monitoring / Security
Published report; unit tests on the spell-reconstruction logic, which is where the bugs actually hide; a model card documenting intended use and known failure modes; pseudonymised identifiers.

Evaluation
Baseline comparison, PR-AUC, calibration, and a decision threshold justified by the cost of a false positive (a wasted discount) against a false negative (a lost customer). Convert the confusion matrix into pounds.

Scalability / GitHub / README
Standard structure. `/docs/model-card.md` and `/docs/exec-brief.pdf` are the differentiators.

Build time / difficulty
2 weeks · Moderate

CV bullet
"Modelled subscriber churn across [N] acquisition cohorts using Kaplan–Meier survival analysis and a calibrated gradient-boosted classifier, converting the confusion matrix into expected pounds to set an intervention threshold that improved projected cost-per-retained-customer by [X]% against a blanket-discount baseline."

Interview questions
Why survival analysis rather than a classifier? Why a temporal split? Your model is 87% accurate — why is that number meaningless here? How did you choose the threshold? What would you tell marketing if they asked you to prove the

intervention worked?

Plausible UK users

Reasonable inference: telecoms, retail banking and insurance, subscription media, gyms and leisure, SaaS. Insurance renewals in particular employ a large number of UK junior analysts.

P4 — Automated Management Information Pipeline Data Analyst 85/100

Business problem
An operations team spends roughly two days every month assembling an MI pack by hand: exporting from three systems, pasting into Excel, refreshing pivot tables, emailing a PDF. The pack is late, occasionally wrong, and the analyst who builds it does no analysis.

Target user
MI Analyst and their manager; the recipients are department heads.

Business process
Month-end close, pack production, distribution, and the inevitable round of corrections.

Pain point
Manual, unauditable, error-prone, and it consumes exactly the capacity that should be spent interpreting the numbers.

Why this matters
This is the most common actual job description behind the titles MI Analyst and Reporting Analyst, and automating it is the most common thing such an analyst is asked to do in their first year. It is also the easiest project to make believable, because everyone recognises the pain.

Proposed solution
Replace the process end to end: scheduled extraction, a validated transformation layer with reconciliation checks, a Power BI report with paginated PDF export, and automated distribution — with a before/after time-and-error comparison.

Expected KPI
Pack production time from ~16 hours to under 30 minutes; reconciliation errors caught before distribution rather than after; a stated pounds-per-year figure using a realistic analyst day rate.

Data sources
Three deliberately heterogeneous public sources with different grains, cadences and identifier conventions — for example ONS labour market data, a government spend-over-£25k transparency file, and DfT or Environment Agency operational data. The mismatch is the point of the exercise.

Technical architecture
Python extractors → landing zone → Power Query and SQL transformation with an entity-resolution step reconciling three identifier schemes → PostgreSQL marts → Power BI with paginated export → scheduled distribution.

Stack
Python, SQL, PostgreSQL, Power Query (M), Power BI, DAX, GitHub Actions, Excel for the legacy comparison.

Analytics methodology
Variance analysis against prior period and budget; exception reporting with tolerance bands; automated commentary on material variances.

SQL / Backend / Frontend
Reconciliation queries and an audit table are the core SQL. No API. Power BI plus a paginated PDF that looks like a real MI pack.

Deployment / Testing / Monitoring
Scheduled on GitHub Actions. Tests: source-to-target row reconciliation, control totals, schema-drift detection, and a deliberate failure test proving the pipeline halts rather than publishing wrong numbers. Run-history table capturing duration and row counts.

Security / Scalability
Secrets in encrypted GitHub secrets, never in code — and say so in the README, because juniors get this wrong constantly. Note where this design breaks at scale and what replaces it.

Evaluation
The before/after table: manual steps, elapsed time, error rate, and who is now free to do what instead.

Build time / difficulty
1.5 weeks · Moderate-low

CV bullet
"Automated a three-source management information pack, cutting production from ~16 hours of manual work to a 25-minute scheduled run with source-to-target reconciliation, schema-drift detection and fail-closed error handling; the pipeline halts on control-total mismatch rather than publishing unverified figures."

Interview questions

What happens when a source changes its schema silently? Why fail closed rather than publish with a warning? How do you reconcile three systems with different customer identifiers? Who signs off that the automated pack is correct?

Plausible UK users

Reasonable inference: local authorities, housing associations, NHS trusts, insurance operations, utilities, and any mid-sized business with a finance MI function. This is close to ubiquitous.

P5 — Grounded Regulatory Compliance Assistant AI Engineer 87/100

Business problem
Compliance officers in regulated firms answer the same questions repeatedly by searching a corpus of handbook sections, policy statements and guidance notes running to thousands of pages and changing continuously. Each answer takes 20–40 minutes, and the risk of citing a superseded version is real.

Target user
Compliance analyst in a small or mid-sized regulated firm; secondary audience the Head of Compliance, who carries personal regulatory accountability.

Business process
Inbound query from the business → source search → interpretation → written answer with citation → audit record.

Pain point
Slow, inconsistent between officers, unauditable. A generic chatbot is worse than useless here, because a confident wrong citation creates precisely the risk the function exists to prevent.

Why this matters
RAG appears in 17.78% of UK AI Engineer adverts. Your differentiator is not that you built RAG; it is that you built RAG that refuses to answer when it should.

Proposed solution
A retrieval-grounded assistant over a versioned regulatory corpus that answers only from retrieved passages, always cites section and version, abstains below a retrieval-confidence threshold, and logs every answer for audit.

Expected KPI
Median time-to-answer; citation accuracy; and the headline metric — abstention rate on out-of-corpus questions, which should approach 100%.

Data sources
FCA Handbook and policy statements, ICO guidance, or UK legislation from legislation.gov.uk — all genuinely public and genuinely structured by section and version.

Technical architecture
Structure-aware chunking preserving section hierarchy → embeddings → hybrid retrieval (BM25 plus dense, fused by reciprocal rank) → cross-encoder reranker → grounded generation with enforced citation → abstention gate → audit log. FastAPI in front, thin Next.js interface.

Stack
Python, FastAPI, PostgreSQL with pgvector (justify this over a dedicated vector database: one database, transactional consistency with the audit log, no extra operational surface), Azure OpenAI or Azure AI Foundry, Docker, Azure Container Apps.

AI methodology
Structure-aware chunking; hybrid retrieval; cross-encoder reranking; constrained generation with citation enforcement; a confidence gate driven by reranker score.

Evaluation — the core of the project
Hand-build a gold set of 80–120 questions with known correct source sections, including 20 deliberately unanswerable ones. Measure retrieval recall@5 and @10, MRR, citation precision, faithfulness (does every claim trace to a retrieved passage), abstention rate on the unanswerable set, p50 and p95 latency, and cost per query in pence. Compare three configurations — dense only, hybrid, hybrid plus rerank — and publish the table. Include a failure taxonomy of 15 real failures you classified by hand.

Backend / Database / Frontend
FastAPI with an OpenAPI spec, API-key auth and rate limiting. PostgreSQL plus pgvector for both vectors and audit. A minimal Next.js chat interface showing citations inline.

Deployment / Testing / Monitoring
Dockerised, deployed to Azure Container Apps. Pytest suite including retrieval regression tests that fail CI if recall@5 drops. Structured logging, a metrics endpoint, and a small dashboard showing queries, latency, abstentions and cumulative spend.

Security
Prompt-injection handling: retrieved documents are treated as data, never as instructions — and you prove it with a test case containing an injected instruction planted in the corpus. No query content in third-party logs. A written data-handling policy.

Cost
Publish actual pence-per-query and projected monthly cost at 1,000 queries a day. Almost no junior does this, and every hiring manager cares.

Scalability
Embedding cache, index growth, and the point at which pgvector should give way to a dedicated index.

Build time / difficulty
3 weeks · High

CV bullet
"Built a citation-grounded regulatory assistant over [N] sections of FCA Handbook text; hybrid retrieval with cross-encoder reranking reached recall@5 of [X]% against a hand-built 100-question gold set, with [Y]% abstention on out-of-corpus queries, p95 latency of [Z]ms and a measured cost of [N]p per query."

Interview questions
Why hybrid retrieval rather than dense alone — what did the numbers say? How do you know it is not hallucinating? What happens when a document in your corpus contains "ignore previous instructions"? Why pgvector? What is your abstention threshold and how did you pick it? What breaks at 10,000 documents?

Plausible UK users
Reasonable inference: financial services compliance, insurance, legal services, pharmaceutical regulatory affairs, public-sector policy teams. *Evidence:* "Security Cleared" appears in 6.27% of UK AI Engineer adverts, indicating substantial regulated and government AI demand.

P6 — Document Intelligence Extraction Service AI Engineer 84/100

Business problem
A finance shared-service team keys supplier invoices into an ERP by hand. Volume runs to a few thousand a month, the error rate is around 2–3%, and every error becomes a payment query that costs more to resolve than the original keying.

Target user
Accounts Payable Manager; secondary audience the systems team who must integrate it.

Business process
Invoice arrives → manual key into ERP → three-way match against purchase order and goods receipt → approval → payment.

Pain point
Slow, expensive per document, and the error rate creates downstream rework and supplier friction.

Why this matters
This is the most commercially concrete AI use case in the UK mid-market and the one most likely to come up in an AI Solutions consultancy interview. It also forces you to solve structured-output reliability, which is the real engineering problem in applied LLM work.

Proposed solution
An API that accepts a document and returns a validated, typed extraction with per-field confidence, routing low-confidence fields to human review rather than guessing.

Expected KPI
Straight-through-processing rate; field-level accuracy; cost per document against a manual keying baseline; and critically, the error rate on auto-approved documents.

Data sources
Public invoice and receipt datasets, plus a synthetic generator producing realistic UK invoices with genuine variation — different layouts, VAT treatments, multi-page documents, poor scans, handwriting in the margin. Generate the awkward cases deliberately.

Technical architecture
Upload → queue → document parse → multimodal or OCR-plus-LLM extraction against a Pydantic schema → validation layer (VAT arithmetic must reconcile, dates must be plausible, line items must sum to the total) → confidence scoring → auto-approve or route to review → webhook callback.

Stack
Python, FastAPI, Pydantic, PostgreSQL, Azure Document Intelligence and Azure OpenAI, Docker, Azure Container Apps, a lightweight queue.

AI methodology
Structured output with schema enforcement and retry-on-validation-failure; a business-rules layer that catches what the model gets arithmetically wrong; confidence derived from both model signal and rule agreement.

Evaluation
A 200-document labelled test set. Field-level precision and recall per field type. A baseline comparison against Azure Document Intelligence prebuilt models alone, so you can show what the LLM layer adds — and where it does not help. Latency distribution, cost per document, and a failure gallery of the ten worst extractions with your analysis of each.

Backend / Database / Frontend
FastAPI with versioned endpoints, OpenAPI docs, idempotency keys, async processing with status polling. PostgreSQL for documents, extractions and review decisions. A small Next.js review queue — this is where your React skills earn their place.

Deployment / Testing / Monitoring
Docker, Azure Container Apps, GitHub Actions CI running the eval set on every merge and failing the build on accuracy regression. Metrics: throughput, p95 latency, STP rate, cost, review-queue depth.

Security
Documents contain supplier bank details. Encrypt at rest, redact from logs, define retention, and write the policy down — demonstrating you thought about it before anyone asked.

Scalability
Queue-based horizontal scaling; the cost curve at 100k documents a month, and where a fine-tuned smaller model would beat the general one.

Build time / difficulty

2.5 weeks · High

CV bullet

"Built a containerised FastAPI document-extraction service reaching [X]% straight-through processing on a 200-document labelled test set, with schema-enforced structured output, an arithmetic validation layer, per-field confidence routing to human review, and CI that blocks merges on accuracy regression."

Interview questions

How do you stop the model returning malformed JSON? What is your confidence score actually measuring? Why route to human review rather than improve the prompt? What did the LLM layer add over the prebuilt model — be specific. How would you handle a supplier who redesigns their invoice?

Plausible UK users

Reasonable inference: accountancy and BPO firms, insurance claims handling, logistics documentation, NHS procurement, construction subcontractor payment applications.

P7 — Service Desk Triage with Guardrails and Human Fallback AI Engineer 80/100

Business problem
An IT service desk receives several hundred tickets a day. Triage — categorising, prioritising, routing — is done by first-line staff, takes three to five minutes per ticket, and misroutes roughly one in six, which adds a full handover cycle to resolution time.

Target user
Service Desk Manager, measured on first-time-fix rate and mean time to resolution.

Business process
Ticket raised → triage → queue assignment → resolution → closure.

Pain point
Triage consumes skilled time, and misrouting compounds into SLA breaches.

Why this matters
"Workflow" appears in 10.20% of AI Engineer adverts and Intelligent Automation Engineer is a real UK title. More importantly, this is where you demonstrate the judgement to *not* use an LLM.

Proposed solution
A triage service that classifies, prioritises and routes behind a confidence gate with mandatory human fallback — using a cheap classical classifier for the routine majority and reserving LLM calls for the ambiguous remainder.

Expected KPI
Misroute rate against the human baseline; triage time per ticket; percentage handled without human touch; cost per ticket.

Data sources
Public IT support ticket datasets, augmented with a synthetic generator producing realistic UK-context tickets including the awkward ones — multi-issue tickets, tickets that are really complaints, tickets written in two lines of frustration.

AI methodology — the key decision
Build a TF-IDF plus gradient-boosting baseline first and measure it. Then build the LLM version and measure it. Then build the hybrid router that escalates only low-confidence cases. Publish all three with accuracy and cost. The finding — that the cheap model handles most traffic at a fraction of the cost — is the most senior thing in your entire portfolio, because it shows you optimise for the business rather than for the interesting technology.

Guardrails
Confidence threshold with human fallback; a hard daily spend ceiling that degrades gracefully to the classical model when hit; PII redaction before any external call; refusal to auto-close anything; a full decision audit log.

Stack / Backend / Database
Python, scikit-learn, FastAPI, PostgreSQL, Azure OpenAI, Docker, Azure Container Apps.

Evaluation
Three-way comparison (baseline / LLM / hybrid) across accuracy, macro-F1, cost per 1,000 tickets and p95 latency; a confusion matrix showing which categories the system confuses and why; a written note on the categories where it should never be trusted.

Testing / Monitoring / Security / Scalability
Pytest regression suite on the eval set; monitoring of category drift over time, which is the strongest MLOps signal you can show cheaply; PII redaction tested explicitly; queue-based scaling.

Build time / difficulty
2 weeks · Moderate-high

CV bullet
"Built a hybrid ticket-triage service routing 71% of tickets through a TF-IDF/gradient-boosting classifier and escalating only low-confidence cases to an LLM, matching pure-LLM accuracy within [X] points at [Y]% of the cost, with a hard spend ceiling, PII redaction and mandatory human fallback."

Interview questions
When should you not use an LLM? How did you set the confidence threshold? What happens when your spend ceiling is hit at midday? How do you detect that ticket categories have drifted? Why is macro-F1 the right metric here?

Plausible UK users
Reasonable inference: managed service providers, local government contact centres, utilities customer service, university IT services, NHS digital service desks.

P8 — LLM Evaluation and Regression-Testing Harness AI Engineer 89/100

Business problem
A company has shipped three LLM features. Nobody can say whether last week's prompt change made things better or worse, whether the model provider's silent update degraded output, or how much any of it costs. Changes ship on instinct and regressions are discovered by customers.

Target user
Engineering lead responsible for AI features; secondary audience the finance owner asking why the API bill moved.

Business process
Prompt or model change → deploy → hope → user complaint → investigate.

Pain point
No baseline, no regression detection, no cost attribution. This is the actual state of most UK companies shipping LLM features today.

Why this matters
Section 8 identified this as the widest gap between demand and junior supply. MLOps appears in 9.54% of adverts, CI/CD in 7.97%, observability in 7.06% — and essentially no junior portfolio addresses any of them. This is your highest-leverage AI project despite its lower raw demand score.

Proposed solution
A harness that runs a versioned evaluation suite against any LLM-backed endpoint on every commit, produces a scorecard, blocks merges on regression, and tracks quality and cost over time.

Expected KPI
Regressions caught before release; time to detect a provider-side degradation; per-feature weekly cost made visible.

Data sources
Your own P5, P6 and P7 endpoints. This project is the connective tissue of your AI portfolio, which is itself a strong signal — it shows you think in systems rather than repositories.

Technical architecture
Eval suite as versioned YAML → runner executing against a target endpoint → graders (exact match, semantic similarity, rubric-based LLM-as-judge) → results store → scorecard and trend dashboard → GitHub Actions gate.

Stack
Python, pytest, FastAPI for the results service, PostgreSQL, GitHub Actions, a simple dashboard.

AI methodology
Multiple grader types with a stated rationale for each; an LLM-as-judge calibrated against 100 human-labelled examples with the agreement rate reported honestly — if your judge agrees with you 78% of the time, publish 78%; and statistical significance testing so you do not chase noise on a 50-case set.

Evaluating the evaluator
Inter-rater agreement between the judge and your human labels; a sensitivity analysis showing how many cases you need to detect a five-point regression; the false-alarm rate.

Backend / Database / Frontend
FastAPI results API; PostgreSQL for runs, cases and scores; a dashboard showing score trend, cost trend and per-case history.

Deployment / Testing / Monitoring / Security / Scalability
Runs in CI. Tested on itself. Tracks its own cost. Redacts eval data in logs. Parallelised case execution with concurrency limits.

Build time / difficulty
2 weeks · High

CV bullet
"Built a CI-integrated LLM evaluation harness gating merges on quality regression across three production-style AI services; calibrated an LLM-as-judge grader against 100 human labels at [X]% agreement and used a sensitivity analysis to size eval sets for detecting a five-point regression at [Y]% power."

Interview questions
How do you know your judge is right? How many eval cases do you need? What do you do when the model provider updates silently? How do you stop the eval set leaking into the prompt? What does running this on every commit cost, and is it worth it?

Plausible UK users

Reasonable inference: any UK firm with LLM features in production — fintech, legal tech, healthtech, consultancies delivering AI to clients. *Evidence:* observability and MLOps together appear in over 16% of UK AI Engineer adverts.

P9 — Governed Natural-Language Analytics Layer Hybrid 91/100

Business problem
Business users queue for the analytics team to answer questions that are, technically, one query away. The analytics team spends its week on ad-hoc requests instead of analysis. Self-service BI was supposed to fix this and did not, because users cannot navigate the semantic model.

Target user
Category or operations manager with no SQL; secondary audience the analytics lead whose backlog this clears and who will be sceptical of it.

Business process
Question → ticket to analytics → two-day wait → answer → follow-up question → repeat.

Pain point
Analyst capacity consumed by trivia and decisions delayed by days — while the naive fix, unrestricted text-to-SQL, is dangerous, because a plausible wrong number is worse than no number at all.

Why this matters
This is the exact intersection of your two tracks. It requires SQL, dimensional modelling and KPI governance on one side, and retrieval, structured generation and evaluation on the other. No other single project demonstrates both as convincingly.

Proposed solution
A natural-language interface over the P2 retail warehouse that generates SQL *only* against a curated semantic layer of certified metric definitions, validates every query before execution, shows the user the SQL and the metric definition it used, and refuses questions it cannot answer safely.

Expected KPI
Percentage of ad-hoc requests self-served; execution accuracy against a gold query set; and the metric that matters most — the rate of confidently wrong answers, driven towards zero.

Data sources
The P2 PostgreSQL warehouse. Reusing it is deliberate: it demonstrates portfolio coherence and saves three days of setup.

Technical architecture
Semantic layer in YAML (entities, metrics with certified SQL expressions, allowed joins, synonyms) → question → retrieval over the semantic layer to select relevant metrics and dimensions → constrained SQL generation restricted to the allowed surface → static validation (read-only, no cross-grain joins, row limit, cost estimate) → execution → result plus SQL plus metric definition plus caveat.

Stack
Python, FastAPI, PostgreSQL, sqlglot for SQL parsing and validation, Azure OpenAI, Next.js, Docker.

Analytics methodology
A properly governed metric catalogue: every metric has a name, formula, grain, owner, refresh cadence and documented caveat. This artefact alone is worth showing in a Data Analyst interview, independently of the AI.

AI methodology
Schema and metric retrieval rather than dumping the whole schema into context; constrained generation; self-correction on validation failure with a retry limit; abstention when the question maps to no certified metric.

Evaluation
A gold set of 60–80 natural-language questions with hand-written correct SQL and expected results. Measure execution accuracy — does the result match — not string similarity. Break results down by difficulty (single metric, filtered, multi-dimension, requiring a join, requiring a time comparison), because the accuracy cliff between categories is the interesting finding. Report abstention rate and, separately, the confidently-wrong rate. Compare against a naive baseline that dumps the raw schema, to quantify what the semantic layer buys.

Backend / Database / Frontend
FastAPI with query history and feedback capture. PostgreSQL with a read-only role and statement timeout — defence in depth, not just instructions in a prompt. Next.js interface showing answer, SQL, metric definition and a "this looks wrong" button.

Deployment / Testing / Monitoring / Security
Docker on Azure Container Apps. The gold set runs in CI. Monitoring of accuracy, abstention, latency and cost. Security is central: read-only role, SQL parsed and validated rather than trusted, injection tests in the suite, row limits enforced at the database.

Scalability
Semantic-layer growth, result caching, and the point at which a dedicated semantic-layer product earns its licence.

Build time / difficulty
3 weeks · High

CV bullet
"Built a governed natural-language analytics layer over a retail warehouse: constrained SQL generation against a certified metric catalogue with parse-level validation and a read-only role, reaching [X]% execution accuracy on an 80-question gold set versus [Y]% for a raw-schema baseline, with [Z]% abstention and zero confidently-wrong answers on the held-out set."

Interview questions
Why a semantic layer rather than giving the model the schema? How do you stop it writing a DELETE? What is your confidently-wrong rate and why is that the metric that matters? Where does accuracy fall off, and what would you do about it? Would you let a director make a pricing decision on this output — why or why not?

Plausible UK users
Reasonable inference: retail and consumer goods, insurance MI functions, logistics, banking commercial teams. *Evidence:* AI appears in 47.34% of UK Data Analyst adverts, which is the demand signal for exactly this capability.

P10 — ImputeIQ: Missing Data Diagnosis and Imputation Platform 96/100 — FLAGSHIP

Hybrid · Flagship

Business problem

Analysts across every regulated industry make silent, undocumented decisions about missing data. The default is `dropna()` or a mean fill, applied without testing why the data is missing. When missingness is related to the outcome, both choices produce biased results that look entirely normal — and that analysis then informs a clinical, financial or regulatory decision with no record that a judgement was ever made.

Target user

Data analyst or statistician preparing an analysis dataset; secondary audience the reviewer or regulator who must later be satisfied the handling was defensible.

Business process

Dataset received → cleaning → missing values handled (usually silently) → analysis → report → decision.

Pain point

The handling step is invisible, untested and unauditable — and it is the step most likely to invalidate everything downstream of it.

Why this matters

Data quality appears in 10.53% of UK Data Analyst adverts and validation in 3.83%, yet almost no candidate can discuss missingness beyond dropping rows. This project lets you say something true and rare: that you know when a mean imputation is a lie.

Proposed solution

A platform that profiles missingness, statistically diagnoses the mechanism (MCAR, MAR, MNAR), routes each column to an appropriate imputation method based on that diagnosis and the column's semantic type, quantifies how much the conclusions depend on that choice through sensitivity analysis, benchmarks against synthetically injected missingness where ground truth is known, and generates a plain-English explanation and audit report.

Expected KPI

Time from raw dataset to a defensible, documented analysis set; bias reduction against ground truth on the synthetic benchmark (RMSE, MAE, KS distance versus naive mean imputation); and the proportion of imputation decisions carrying a recorded statistical justification — from 0% to 100%.

Data sources

Your CPRD Aurum observation dictionary work provides the clinical grounding. For the public demonstration, use openly available health and survey data (NHS open data, UK Data Service extracts, UCI clinical sets) plus the synthetic missingness injection engine, which creates ground truth by construction.

Technical architecture

Next.js 16 dashboard → FastAPI backend over HTTP and Socket.IO → pipeline orchestrator → column semantics inference → missingness profiling → mechanism diagnosis (Little's MCAR test, correlation and distribution-shift tests) → method router → imputation engine (mean/median/mode, KNN, MICE, MissForest) → sensitivity engine → synthetic benchmark engine → LLM explainer against a constrained explanation schema → PDF audit report. Long-running jobs stream progress over WebSockets.

Stack

Python 3.12, FastAPI, pandas, NumPy, scikit-learn, statsmodels, Socket.IO, PostgreSQL, Next.js 16 with React 19, Redux Toolkit, Tailwind, an LLM for explanation generation, Docker.

Statistical methodology

Little's MCAR test; pairwise correlation of missingness indicators against observed variables to evidence MAR; distribution-shift metrics to flag suspected MNAR; multiple imputation by chained equations with Rubin's rules for pooling; MissForest for non-parametric cases; and explicit acknowledgement that MNAR cannot be confirmed from the observed data alone, only suspected — which is the single most statistically literate statement in your whole portfolio.

AI methodology

The LLM is confined to translation, never inference. It receives structured diagnostic output and emits an explanation conforming to a fixed schema. It never chooses a method. Document this boundary explicitly — it is a governance decision, and articulating it well will impress a senior interviewer more than any model would.

SQL / Backend / Database

FastAPI with typed schemas, job orchestration and WebSocket progress. PostgreSQL for datasets, jobs, diagnostics, decisions and the audit trail. SQL for audit and lineage queries.

Frontend
Tabbed workflow: overview, diagnosis, imputation, sensitivity, validation gate, approval. An override modal lets a user reject the router's recommendation — with a mandatory reason, which is then recorded. That single design decision demonstrates more product judgement than most junior portfolios contain in total.

Deployment
Docker Compose for the full stack; deployed to Azure Container Apps with the frontend on Azure Static Web Apps. **Deploying this to Azure is the single highest-value action in your twelve-week plan**, because it converts your largest skill gap into demonstrated evidence on your strongest asset.

Testing
You already have a meaningful suite — routing and diagnosis, imputation and sensitivity, multiple imputation, cleaning rules, action normalisation, data dictionary. Add property-based tests asserting that imputation preserves column dtype and never introduces values outside the observed support; a golden-dataset regression test; and CI running all of it with a visible coverage badge.

Monitoring
Job duration and failure rate; per-method runtime (MICE and MissForest are slow, and you should be able to say by how much); LLM token spend per report.

Security
This is health-adjacent data, so the handling policy is part of the product: no raw record values in LLM prompts (send summary statistics only), configurable retention, an audit log of every access. Write it up as `/docs/data-handling.md`.

Evaluation
The synthetic benchmark is the evaluation, and it is genuinely rigorous: inject known MCAR, MAR and MNAR patterns at 5%, 15%, 30% and 50% missingness into complete datasets, then measure RMSE, MAE and Kolmogorov–Smirnov distance for each imputation method against the known truth. Publish the resulting matrix as the headline artefact. It answers, with evidence, the question every analyst waves away: *does the choice of method actually matter?*

Scalability
MICE and MissForest are computationally expensive; document the row and column limits at which each becomes impractical, and what replaces them (chunked imputation, sampling for diagnosis, a queue with workers).

GitHub structure
`/backend/app/core` (the engines), `/backend/tests`, `/frontend`, `/docs/methods-reference.md` (you already have this — it is a genuine asset), `/docs/benchmark-results.md`, `/docs/data-handling.md`, `/docs/architecture.png`, `/docs/exec-brief.pdf`.

README
Rewrite the opening. It currently leads with badges and feature bullets. It should lead with two sentences of problem — what silent mean-imputation does to a conclusion, and what this prevents. Then the benchmark matrix. Then the screenshot. Then the live link. Badges and feature lists can wait until the reader is convinced they should care.

Demo
A live deployed instance with a pre-loaded example dataset, plus a five-minute video walking one dataset from upload to audit report, narrated as if to a clinical researcher rather than an engineer.

Build time / difficulty
2 weeks of remaining polish, deployment and documentation · High (already substantially built)

CV bullet
"Designed and built ImputeIQ, a full-stack missing-data platform (FastAPI, PostgreSQL, Next.js 16) that diagnoses missingness mechanism via Little's MCAR test and distribution-shift analysis, routes columns to MICE, MissForest or KNN imputation accordingly, and quantifies conclusion stability through sensitivity analysis; benchmarking against synthetically injected missingness at four rates showed [X]% lower RMSE than mean imputation under MAR conditions."

Interview questions
What is the difference between MAR and MNAR, and can you prove which one you have? Why does mean imputation understate variance? What are Rubin's rules and why do you need them? Why does the LLM not choose the method? Your router picked MICE for a column with 60% missingness — defend that. What breaks at a million rows? How would you convince a clinician who has always used complete-case analysis?

Plausible UK users
Evidence: CPRD is a real UK primary-care research database with documented missingness challenges, and clinical research routinely requires justified missing-data handling. *Reasonable inference:* NHS analytics and research teams, clinical research

organisations, pharmaceutical biostatistics, insurance actuarial functions, ONS and government statistical services, academic research groups.

13. Why P10 is the flagship

ImputeIQ takes the flagship slot for four reasons, in order of importance.

It cannot be exhausted in ten minutes. Every other project has a bottom. An interviewer can ask five questions about your churn model and reach the edge of it. ImputeIQ has a statistical layer, a routing layer, an engineering layer, a governance layer and a product layer, and each supports twenty minutes of genuine discussion. It is the only item in your portfolio that can fill a full technical interview on its own — which matters enormously when you have no work experience to talk about instead.

It demonstrates the thing that is hardest to fake: statistical judgement. Anyone can call an LLM API. Very few junior candidates can explain why mean imputation understates variance, what Rubin's rules do, or why MNAR cannot be established from observed data. That knowledge is the strongest available signal that you are not a bootcamp graduate following a tutorial, and it speaks directly to the 10.53% of analyst adverts asking about data quality.

It spans both tracks without contrivance. Statistics, PostgreSQL and data quality on the analyst side; FastAPI, WebSockets, structured LLM output, Docker and testing on the engineering side. It is not a data project with AI bolted on, nor an AI project with a chart attached. Both halves are load-bearing.

It already exists. You have the engines, the tests and the frontend. The distance between what you have and what a hiring manager needs to see is deployment, benchmark publication and a rewritten README — roughly two weeks, against twelve for something new of comparable depth.

> **The one risk, and how to kill it.** This is your dissertation, and dissertation projects carry a specific smell: exhaustive method comparison, no named user, no deployment, and a README that opens like an abstract. Three changes remove it. First, **name the user in the first paragraph** — a clinical researcher preparing an analysis set, not "the practitioner". Second, **deploy it and put the URL at the top**; nothing says "not coursework" like a working link. Third, **lead with the benchmark result, not the methodology** — that method choice changes RMSE by X% under MAR is a business fact, and the statistics are the evidence for it rather than the point of it. Do not hide that it is your dissertation; academic work that shipped is a strength. Present it as a product with a rigorous foundation, not as research with a user interface.

14. Project-to-skill coverage matrix

Skill	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10
Advanced SQL	●	●●	●	●					●●	●
PostgreSQL	●	●	●	●	●	●	●	●	●	●
Power BI / DAX	●●	●●	●	●●						
Power Query / Excel				●●						
Dimensional modelling	●●	●●		●					●	
Python (analysis)	●	●	●●	●			●			●●
Python (engineering)				●	●●	●●	●●	●●	●●	●●
Statistics	●		●●					●		●●
Forecasting / projection	●●		●							
Machine learning			●●				●●			●
LLM application design					●●	●●	●	●	●●	●
RAG / retrieval					●●				●	
AI evaluation			●		●●	●●	●●	●●	●●	●●
FastAPI / REST					●●	●●	●●	●	●●	●●
Docker					●	●	●		●	●
Azure deployment	●				●●	●●	●		●	●●
CI/CD		●		●●	●	●●	●	●●	●	●●
Testing	●	●●	●	●●	●	●●	●	●●	●●	●●
Monitoring / cost				●	●●	●●	●●	●●	●	●
Security / governance	●		●	●	●●	●●	●●	●	●●	●●
Stakeholder writing	●●	●●	●●	●	●	●	●	●	●●	●●
React / Next.js					●	●			●	●●

●● = primary demonstration · ● = present and defensible. Every Tier 1 skill from section 9 is covered at primary level at least twice, and no two projects lead on the same combination.

15. Final ranking

Rank	Project	Score	Why it sits here
1	P10 ImputeIQ	96	Unmatched depth, spans both tracks, statistically literate, and 80% built. The only project that can carry a full interview alone.
2	P1 NHS RTT analytics	91	Maximum demand alignment. Public-sector analytics is the largest junior employer in your region and the data is genuinely open. Your most *employable* project, even though it is not your deepest.
3	P9 NL analytics layer	91	Ties on score, ranks below P1 on feasibility and above it on differentiation. The clearest single proof that you belong in both markets.
4	P2 Revenue leakage	90	Your SQL showcase and the archetypal commercial analyst brief. Sits below P1 only because it leans on synthetic cost data.
5	P8 Eval harness	89	Lowest raw demand of the AI set, highest scarcity. The project most likely to make an AI interviewer sit up.
6	P5 Compliance RAG	87	Matches the 17.78% RAG signal directly, and the abstention behaviour separates it from every other portfolio RAG.
7	P4 MI automation	85	The best recognition-to-effort ratio here. Every hiring manager has lived this pain.
8	P6 Document extraction	84	The most commercially concrete AI project, but a crowded space and less differentiated than P8.
9	P3 Churn cohorts	81	Solid, expected, well-trodden. Included because the statistical rigour differentiates even where the topic does not.
10	P7 Triage automation	80	Good judgement demonstration, weakest differentiation. First to cut.

Part 3 — Execution

16. The twelve-week plan

This assumes about 30 productive hours a week, not 80. If you have less, cut projects from the bottom of the ranking rather than doing ten things badly. The plan front-loads the two things that unblock everything else: Power BI, because it is your largest employable gap, and deploying ImputeIQ, because it converts work you have already done into evidence.

> **You start applying in week 3, not week 12.** By the end of week 2 you will have one deployed flagship and one published dashboard. That is already a stronger portfolio than most applicants have at any point. Applications take four to eight weeks to convert, so an application sent in week 3 interviews in week 7 — by which time you have six projects. Waiting until everything is finished wastes the entire pipeline lead time and is the most common self-inflicted error in a job search.

Week	Build	Learn	Career actions
1	ImputeIQ: rewrite README around the problem; run the full synthetic benchmark at 5/15/30/50% and publish the matrix; add property-based tests; get CI green with a coverage badge.	Azure fundamentals: resource groups, Container Apps, Static Web Apps, managed identity. Start DP-900.	Buy a domain. Set up a one-page portfolio site. Rewrite LinkedIn headline.
2	Deploy ImputeIQ to Azure Container Apps + Static Web Apps. Record the five-minute demo. Write `data-handling.md` and the exec brief.	Power BI from zero: data model, relationships, measures vs columns, DAX basics (CALCULATE, filter context).	Sit DP-900. CV draft A. Connect with 20 Manchester/Liverpool analysts and recruiters.
3	**P1 NHS RTT**: ingestion, star schema, first Power BI pages.	DAX time intelligence; row-level security; star schema theory.	**Start applying.** 8–10 tailored Data Analyst applications. Post ImputeIQ on LinkedIn.
4	P1 finish: breach-risk indicator, backtest, data-quality page, publish, exec brief.	SQL window functions to depth; NHS RTT domain rules.	10 applications. First recruiter conversations. Refine CV A on feedback.
5	**P2 Revenue leakage**: synthetic cost/promo generator, star schema, the margin bridge.	Price-volume-mix theory; advanced SQL patterns (gaps and islands, recursive CTEs).	10 applications. Post P1 with a screenshot. Ask two analysts for a 20-minute call.
6	P2 finish: reconciliation tests, Power BI waterfall, KPI catalogue, exec brief.	Power Query (M) for P4; Tableau basics — rebuild one P1 page only.	10 applications. Begin SQL interview drilling (StrataScratch or equivalent, 5 problems a day).
7	**P4 MI automation** — the fastest build here, deliberately placed as a breather week.	GitHub Actions scheduling and secrets; schema-drift detection patterns.	10 applications. Expect first interviews. Rehearse the ImputeIQ walkthrough aloud, timed.
8	**P5 Compliance RAG**: corpus ingestion, hybrid retrieval, build the 100-question gold set by hand.	Azure AI Foundry / Azure OpenAI; pgvector; reranking.	CV draft B (AI variant). Start 4–5 AI applications a week alongside the analyst pipeline.
9	P5 finish: abstention gate, evaluation table across three configurations, deploy to Azure, cost page, failure taxonomy.	Prompt injection defence; retrieval metrics (recall@k, MRR, faithfulness).	12 applications across both pipelines. Post the P5 evaluation table — this is your best content.
10	**P9 NL analytics layer**: semantic layer YAML, metric catalogue, constrained generation, sqlglot validation.	sqlglot; semantic layer design; text-to-SQL evaluation methodology.	12 applications. Interview practice: system design out loud, whiteboard-style.
11	P9 finish: 80-question gold set, baseline comparison, deploy. **P8 eval harness** started — it wraps P5 and P9, so it is cheaper now than earlier.	LLM-as-judge calibration; significance testing on small eval sets.	12 applications. Second-round interviews likely. Update CV with real numbers from finished projects.
12	P8 finish and wire into CI for P5 and P9. Portfolio site polish: every project one click from the homepage.	Consolidation. Re-read every README as a stranger would.	12 applications. Full mock interview for both tracks. Plan P6, P3, P7 for weeks 13–18.

Note on scope. Twelve weeks at 30 hours delivers six projects to a high standard (P10, P1, P2, P4, P5, P9) plus P8 substantially complete. It does not deliver ten. I have said this plainly rather than drawing a schedule that looks complete and is not. P6, P3 and P7 belong in weeks 13–18, and by then you may be employed — which is the point. Six excellent projects with a job offer beats ten finished projects without one.

17. Application strategy

Pipeline A — Data Analyst (65% of effort)

Element	Detail
Titles to target	Data Analyst, Junior Data Analyst, Graduate Data Analyst, MI Analyst, Reporting Analyst, Business Intelligence Analyst, Insight Analyst, Performance Analyst, Data & Reporting Analyst, Commercial Analyst, Operations Analyst
Titles to deprioritise	Senior anything; Data Scientist (different bar, PhD-heavy); Data Engineer (real but a different portfolio); Business Analyst without "data" (process and requirements work, not analysis); Analytics Engineer (usually wants dbt plus commercial experience)
Experience bars worth ignoring	Apply at "1–3 years" and "2+ years" — these are aspirational and routinely filled by strong juniors. Apply at "3–5 years" only with a covering note that names the specific gap and the specific evidence. Skip "5+ years".
Volume	10–12 per week, of which 6–8 genuinely tailored. Tailoring means changing the summary line and reordering projects to match their sector — twenty minutes, not two hours.
Highest-yield sectors	NHS trusts and ICBs (use NHS Jobs directly — it is a separate market most applicants ignore); local government; housing associations; universities; insurance; utilities; retail and logistics head offices in Manchester and Warrington
Recruiter outreach	Datatech Analytics, Harnham, Forsyth Barnes, Sanderson, and regional North West agencies. Register with four, speak to two properly, and send them the portfolio link rather than only the CV.
Networking	Manchester data meetups; the Power BI user group; NHS-R community. Two conversations a week, asking about the work rather than about vacancies.

Pipeline B — AI Engineer / AI Solutions (35% of effort)

Element	Detail
Titles to target	AI Solutions Engineer, Applied AI Engineer, AI Developer, Junior AI Engineer, AI Automation Engineer, Graduate Software Engineer at AI-focused firms, Data & AI Consultant, Python Developer at AI companies
Titles to deprioritise	Machine Learning Engineer (usually wants an MSc plus commercial ML); Research Engineer; LLM Engineer at frontier labs; anything at £90k+, which is 50% of the market and not available to you yet
The realistic door	**Consultancy graduate and academy schemes.** Capgemini, Accenture, Sparta Global, BJSS, Kubrick, Version 1, Infosys and similar hire juniors into AI-adjacent roles at £25–35k and train them. This is close to the only structured entry into UK AI engineering, and the salary looks poor against a £87,500 median precisely because the median is senior. Take the pay cut; the second job pays for it.
Volume	4–5 per week, all tailored. Volume does not work here — there are too few genuine junior roles to spray.
Where to look	Employer career pages directly; Gradcracker (99 live AI/ML graduate opportunities from 27 employers at time of research); Bright Network; targetjobs; LinkedIn with the experience filter set to entry level and the salary filter off
The Azure lever	With Azure in 57.52% of these adverts, an AI-102 or AZ-900 certification plus a deployed Azure project moves you past a large fraction of self-taught applicants. Nothing else you can do in three months has this ratio of effort to visibility.

Weekly rhythm

Monday: search and shortlist, 90 minutes. Tuesday and Thursday: five applications each, 2 hours per session. Wednesday: one networking conversation plus LinkedIn activity. Friday: follow up on anything sent more than ten days ago, and log outcomes. Track everything in a spreadsheet with role, company, date, source, tailoring done, and outcome — after six weeks that sheet tells you which channel actually works, and you should reallocate towards it ruthlessly.

18. CV strategy

Two variants, one page each until you have experience to justify two. Same facts, different ordering and vocabulary. Never send the wrong one; the mismatch is obvious and reads as spray-and-pray.

Section	CV A — Data Analyst	CV B — AI Engineer / AI Solutions
Summary	"Analyst with a statistics-heavy MSc and a portfolio of end-to-end analytics work: SQL modelling, Power BI semantic models and automated reporting pipelines built on public NHS and retail data, each delivered with written recommendations for a named business owner."	"Python engineer building and evaluating production-shaped AI systems: retrieval-grounded assistants, structured extraction APIs and CI-gated LLM evaluation, deployed on Azure with measured accuracy, latency and cost per query."
Skills order	SQL · Power BI (DAX, Power Query) · Python (pandas) · Excel · PostgreSQL · Azure · statistics · data modelling · stakeholder reporting	Python · FastAPI · LLM applications (RAG, structured output, evaluation) · Azure AI · PostgreSQL/pgvector · Docker · CI/CD · SQL · TypeScript/React
Project order	P1 NHS → P2 margin → P10 ImputeIQ → P4 MI automation	P10 ImputeIQ → P5 compliance RAG → P9 NL analytics → P8 eval harness
ImputeIQ framing	Lead on data quality, statistical diagnosis and audit trail. Mention the stack briefly.	Lead on the full-stack architecture, WebSocket job orchestration, constrained LLM output and test suite. Mention the statistics briefly.
ATS keywords to include verbatim	SQL, Power BI, DAX, Power Query, Excel, Python, data visualisation, dashboard, stakeholder, data quality, ETL, data modelling, reporting, KPI, Azure	Python, FastAPI, REST API, LLM, RAG, retrieval-augmented generation, prompt engineering, Azure, Azure OpenAI, Docker, CI/CD, evaluation, MLOps, vector database, machine learning
Education	Prominent — the statistical content is a genuine credential for this track. Name the dissertation topic.	Present but below projects. Employers here care what you shipped.
Remove entirely	Django. "Familiar with" anything. Any technology you would not want to be questioned on for ten minutes. A thirty-item skills wall — it reads as tutorial completion when there is no experience anchoring it.	

Bullet construction

Every bullet follows: *what you built → the technical specifics that prove it → the measured outcome*. The measurement is what separates these from everyone else's.

- "Built an RTT performance report over a PostgreSQL star schema, adding a four-week breach-risk indicator backtested at 71% precision across 12 months of provider-specialty data, with row-level security by provider."
- "Decomposed 12 months of gross margin across 1m transactions into price, volume, mix, discount and returns effects via a reconciling SQL bridge, isolating five leakage sources and recommending an owner-level action for each."
- "Reached 84% recall@5 on a hand-built 100-question gold set using hybrid retrieval with cross-encoder reranking, with 95% abstention on out-of-corpus queries at 0.4p per query."

> **Use real numbers only.** The bracketed placeholders throughout this report exist to be replaced by measurements you actually take. An invented figure will not survive the first competent interviewer, and being caught costs you the offer and the recruiter relationship. If a result is unimpressive, report it and explain what you would do differently — that reads as senior, whereas a suspiciously round number reads as fabricated.

19. GitHub strategy

A recruiter spends 40 seconds on a repository, and most of that on the README before the first scroll. Optimise for that, not for the code.

README template, in order

1. **One sentence on the problem** and whose job it affects. No badges above this line.
2. **A screenshot or the benchmark table.** Something visual within the first screen.
3. **The live link**, prominent.
4. **Three findings or results**, as bullets with numbers.
5. **Architecture diagram.** One image. Draw it in Excalidraw; it takes twenty minutes.
6. **How it works** — the interesting technical decisions, not a file listing.
7. **Evaluation** — what you measured and against what baseline.
8. **Limitations** — genuine ones. This section wins more interviews than any other.
9. **Run it locally** — and it must actually work from a clean clone.

Recruiter GitHub checklist

Check	Standard
Pinned repositories	Exactly six, in ranking order, each with a one-line description that states the problem rather than the stack
Profile README	Two sentences on what you do, plus links to the portfolio site and the three strongest projects
Every pinned repo has	A screenshot or diagram in the first screen; a live link where one exists; a limitations section; a licence
Commit history	Incremental commits over weeks with meaningful messages. Not a single "initial commit" containing 4,000 lines — that pattern reads as copied
Tests	Visible `/tests`, CI badge that is green, coverage badge on at least the flagship
CI/CD	A GitHub Actions workflow on every code repo, even if it only runs lint and tests
API projects	Published OpenAPI docs and an example request/response in the README
Documentation	A `/docs` directory containing the exec brief, the architecture diagram, and the evaluation results
Dead weight	Archive or delete tutorial repos, half-finished experiments, and anything you cannot defend. A clean profile of six is stronger than thirty of mixed quality
Secrets	No `.env` in history. Check with a scanner before you publicise anything — a leaked key in a public repo is a hard no from a security-conscious employer

20. LinkedIn

	Data Analyst positioning	AI Engineer positioning
Headline	"Data Analyst | SQL · Power BI · Python | NHS & commercial analytics portfolio | Manchester / North West"	"AI Engineer | Python · FastAPI · RAG · Azure AI | Building and evaluating production LLM systems"
About	Open with the problem you like solving, not with your degree. Three sentences on what you build, one line each on the top three projects with links, and a closing line naming the roles you are looking for and your location. No inspirational framing.	
Featured	Three items: the flagship live link, the strongest dashboard, and the portfolio site. Swap by track as your focus shifts.	
Posting	One post per finished project, roughly fortnightly. Structure: the problem, one screenshot or table, the single most interesting technical decision, the result, the link. No hooks, no thread-bait, no "I'm excited to share". Six good posts over twelve weeks is the right volume.	
Recruiter visibility	Turn on Open to Work (recruiters-only if you prefer discretion). Set your location to Manchester rather than a smaller town — recruiter searches are city-radius based and this alone materially increases inbound.	
Networking	Connect with analysts and engineers doing the job you want, not just recruiters, and send a two-line note referencing something specific about their work. Ask about the role, never for a job. Ten connections a week.	

21. Interview preparation

Data Analyst interviews

Area	Questions to be fluent on
SQL	Difference between RANK, DENSE_RANK and ROW_NUMBER, and when each matters. Find the second-highest value per group. Running total and period-over-period change. Why does a LEFT JOIN plus a WHERE clause on the right table silently become an INNER JOIN? Identify and remove duplicates. Gaps and islands. Explain a query plan you have read.
Business analysis	Revenue fell 8% last month — how do you find out why? A stakeholder asks for a dashboard; what do you ask before building? How do you define an active customer, and who owns that definition? Which metric would you kill?
Statistics	When is the median the honest choice? What is a confidence interval actually saying? How large a sample do you need? Why is a correlation of 0.8 not an argument? What is Simpson's paradox and have you seen it?
Dashboards and BI	Calculated column versus measure. What is filter context and how does CALCULATE change it? Star schema versus a flat table. Import versus DirectQuery. How do you stop a dashboard nobody opens?
Data quality	How do you handle missing data — and this is your question to win, given ImputeIQ. Two systems disagree about the same number; what do you do? How do you catch a silent upstream schema change?
Stakeholders	A director disputes your analysis. What do you do? How do you deliver a finding someone does not want? How do you say no to a request?

AI Engineer interviews

Area	Questions to be fluent on
Python and engineering	Explain your API's error handling. How do you manage dependencies and secrets? Sync versus async and where it matters here. How do you make this idempotent? Walk me through a test you wrote and why.
Architecture	Draw your RAG pipeline. Where is the latency? What fails first under load? Why pgvector rather than a dedicated vector database? How would you add a second corpus?
LLMs and RAG	Chunking strategy and why. Dense versus hybrid retrieval — what did your numbers say? What does a reranker do? How do you enforce citation? What is your abstention threshold and how did you choose it?
Evaluation	How do you know it works? What is your baseline? How do you detect a regression? How do you validate an LLM-as-judge? How many eval cases do you need, and how did you decide?
Production	What do you monitor? What is your cost per request? What happens when the provider has an outage? How do you roll back a prompt change? What is in your logs, and what must never be?
Security	Prompt injection — how do you defend, and how did you test it? What stops this returning another tenant's data? What PII crosses the network boundary?

Hybrid questions — and the answers that separate you

- **"How would AI improve this analytics workflow?"** Point at the ad-hoc query backlog (P9) and at explanation generation (P10). Be specific about which step of a named process gets faster.
- **"When should you not use AI?"** Your P7 finding is the answer: a TF-IDF classifier handled 71% of tickets at a fraction of the cost and comparable accuracy. Also P1, where you chose a linear projection over a model because a clinician has to trust it. Having chosen *against* the interesting technology twice, with data, is a stronger signal than any model you could show.
- **"How would you measure the AI system?"** Never answer "accuracy". Answer with the metric tied to the decision: confidently-wrong rate for P9, abstention rate for P5, straight-through-processing rate with the auto-approved error rate for P6.
- **"What is the ROI?"** Have one costed example ready. P4 is the cleanest: 16 hours a month at a realistic day rate, against roughly £0 of running cost.

- **"How would you validate the output?"** Gold sets, human labels, and the honest agreement rate between your judge and your own labelling.

22. Portfolio red flags to avoid

Red flag	Why it damages you
Tutorial clones and famous datasets	Titanic and Iris tell a reviewer you followed instructions. They cannot distinguish you from anyone else who did.
A chatbot with no evaluation	The most common junior AI project and the most heavily discounted. Without metrics it demonstrates that you can read documentation.
Notebook-as-deliverable	Signals that you have never handed work to anyone who had to use it.
No business problem	Without a named user and a decision, it is a technology demonstration, and technology demonstrations do not get hired.
Fabricated or unbaselined metrics	"40% improvement" over nothing. An interviewer will ask over what, and the answer determines whether they trust anything else you said.
Excessive technology count	Kafka, Kubernetes, Spark, Airflow, three vector databases. Reads as insecurity, and every item is a question you must survive.
No limitations section	Suggests you cannot see the weaknesses in your own work — which is exactly what a reviewer is trying to assess.
Dead demo links	Worse than no link. It proves you did not check.
One giant commit	Reads as copied, whether or not it was.
Ten near-identical projects	Breadth without variety reads as padding.

23. Final decisions

1. What percentage of application effort goes where?

65% Data Analyst, 35% AI Engineer. Not 50/50, because the Data Analyst market has 2.2 times the advertised volume and a 10th-percentile salary of £31,250 against £53,750 — which is the clearest possible evidence of where a junior door exists. Not 90/10, because AI-adjacent analyst roles and consultancy academy schemes are real, and because your AI work is what makes your analyst applications distinctive.

2. Which track gives the strongest probability of entering the UK market?

Data Analyst, clearly. Roughly triple the year-on-year advert growth, genuine trainee and graduate postings at £28–32k, 206 adverts in the North West alone, and a skill profile you already partly hold. AI Engineer is growing faster in rank terms but is growing as a senior title.

3. Which skills to prioritise in three months?

1. **Power BI including DAX and Power Query** — 55.60% of analyst adverts, and currently a total gap. Highest return of anything on this list.
2. **Azure** — 45.66% of analyst adverts and 57.52% of AI adverts, and currently a total gap. The only skill that pays into both tracks at scale. DP-900 then AI-102.
3. **SQL depth** — window functions, CTEs, dimensional modelling. You have the basics; interviews test the next layer.
4. **Written business communication** — one executive brief per project. Cheap to produce, and almost nobody does it.
5. **AI evaluation methodology** — gold sets, baselines, cost per query. Your differentiator in Pipeline B.

Explicitly deprioritise: AWS, Django, deep learning theory, agent frameworks, Kubernetes.

4. Which three projects lead the CV?

CV A: P1 NHS RTT, P2 revenue leakage, P10 ImputeIQ. **CV B:** P10 ImputeIQ, P5 compliance RAG, P9 natural-language analytics.

5. Which project is the flagship?

P10 ImputeIQ — for both tracks, framed differently. It is the only project deep enough to sustain a thirty-minute technical conversation, and it is already 80% built.

6. When do you start applying?

Week 3. Once ImputeIQ is deployed and P1 is published. Applications convert in four to eight weeks, so waiting for a finished portfolio discards the entire pipeline lead time. Your week-3 portfolio already beats most applicants' final one.

7. What could still cause failure?

- **Building instead of applying.** The most likely failure by a distance. Building is comfortable and rejection is not, so the plan drifts towards project eleven. Ten applications a week is a hard floor, in weeks when you feel ready and weeks when you do not.
- **The Power BI gap staying open.** Skip it and you are excluded from 56% of the analyst market regardless of how good the Python is.
- **Depth without communication.** If you cannot explain ImputeIQ to a non-statistician in three minutes, its value collapses at the first interview.
- **Visa or eligibility constraints.** The Sparta Global advert states no sponsorship, which is common at graduate level. If you need sponsorship, filter for licensed sponsors early — it changes which employers are reachable more than any skill gap does.
- **Market conditions outside your control.** Junior hiring is competitive and some of the outcome is timing. Volume and persistence are the only available counters.
- **Applying only in Liverpool.** Merseyside had 49 adverts in six months and a falling median. Manchester plus remote is your real market.

8. What specifically compensates for no internship?

Four things a reviewer can verify in under five minutes: **a deployed, working system with a live URL** (ImputeIQ on Azure); **a published dashboard built on real public data with row-level security and a backtested indicator** (P1); **measured evaluation with a stated baseline and a cost per query** (P5, P9) — which is rarer among juniors than any technology; and **written executive briefs** proving you can hand work to a non-technical person. Collectively these answer the only question that matters when there is no employment history: would this person's output be usable on Monday?

9. If only five projects, which five?

1. **P10 ImputeIQ** — flagship, already built, carries an interview alone.
2. **P1 NHS RTT** — the Power BI and SQL proof, in the sector most likely to hire you.
3. **P9 Natural-language analytics** — the both-tracks proof, and reuses P2's warehouse.
4. **P5 Compliance RAG** — the RAG signal plus the evaluation rigour that distinguishes it.
5. **P2 Revenue leakage** — the advanced SQL and commercial-thinking proof, and a prerequisite for P9.

These five cover every Tier 1 skill, both tracks, and every entry in the experience-deficit framework. P4 is the honourable mention — it is the cheapest to build and the most instantly recognisable to a hiring manager, so add it sixth if you have a spare fortnight.

24. Research base

Item	Detail
Total unique adverts underpinning the quantitative findings	**2,436**
Role split	1,671 Data Analyst · 765 AI Engineer
Supplementary title data	544 "AI Engineering", 1,960 Data Engineer, 31 Gen AI Engineer, 3 Agentic AI Engineer, plus regional breakdowns
Date range	Six months to 10–11 August 2026, with year-on-year comparison to the same periods in 2025 and 2024
Geography	UK-wide, with breakdowns for England, London, UK excluding London, North of England, North West, Manchester, Merseyside, Cheshire, Midlands, South East, Scotland, Wales, and remote
Employment type	Permanent only. Contract roles were excluded as irrelevant to an entry-level search.
Primary source	IT Jobs Watch (itjobswatch.co.uk) — UK permanent IT vacancy index with parsed skill co-occurrence counts
Secondary sources	Individual employer and job-board listings (Sparta Global via targetjobs); Prospects.ac.uk graduate careers guidance; Glassdoor UK (917 Power BI analyst listings); LinkedIn UK volume indicators (1,000+ Power BI Analyst, 3,000+ AI Engineer); Gradcracker (99 live AI/ML graduate opportunities from 27 employers)
Representative listings examined	Sparta Global Junior AI Engineer (London, £25–26k, graduate, no sponsorship); Veolia Energy Data Analyst (City of London, £31,000); ITOL Recruit Data Analyst Trainee (multiple locations, £30–50k); Healthtrust Europe Data Analyst (Birmingham); Yorkshire Water Maintenance Assurance Data Analyst (Bradford); Central London Community Health Trust Sustainability Data Analyst; City Facilities Management Data & Analytics Analyst (Glasgow)

Major patterns

1. Data Analyst demand tripled year on year (454 → 1,671 adverts) and rose 186 rank places.
2. AI Engineer demand rose 351 rank places but remains a predominantly senior title, with a 10th-percentile salary of £53,750.
3. Azure dominates both tracks (45.66% and 57.52%); AWS appears in neither top-30 analyst list and only 11.24% of AI adverts.
4. AI appears in 47.34% of Data Analyst adverts — the two tracks are converging.
5. Python has reached near-parity with Excel in analyst adverts (54.76% against 53.38%).
6. Evaluation, observability and MLOps appear in 7–10% of AI adverts and are almost entirely absent from junior portfolios.
7. Manchester is a rising regional market (£46,750 median, +10%); Merseyside is contracting (−23.81%).

Limitations — read these before acting

- **Title-indexed, so undercounts analytics.** IT Jobs Watch indexes adverts by title within a digital/IT taxonomy. A large volume of Reporting Analyst, MI Analyst and Insight Analyst roles in the NHS, local government and insurance sit outside it. Treat the Data Analyst volume as a floor.
- **Skill co-occurrence is not skill requirement.** A percentage means the term appeared in the advert, not that it was mandatory. The "Microsoft" and "Azure Certification" entries in particular are likely inflated by boilerplate in Microsoft-partner adverts.
- **Salary medians reflect advertised salaries only** — 656 quoted salaries across 1,671 Data Analyst adverts. Adverts without a salary skew towards both the public sector (lower) and senior roles (higher), in unknown proportions.
- **I did not read 100 full job descriptions.** The qualitative characterisation of responsibilities rests on a small number of individual adverts plus the skill-frequency data. Section 6 (the recruiter perspective) and section 13 (the

flagship argument) are reasoned judgement informed by the data, not measurements.

- **Regional figures rest on small samples.** Merseyside's −23.81% median change comes from 49 adverts and could easily be composition noise rather than a real decline.
- **Some sources were inaccessible.** Bright Network returned HTTP 403 and could not be examined; LinkedIn and Indeed volume figures come from search-result summaries rather than from parsing the underlying adverts, so they indicate scale only.
- **All project scores are my judgement, not measurements.** The 100-point scores in section 10 are a structured way of comparing options, and reasonable people would score them differently. The underlying demand percentages they draw on are real; the weightings applied to them are not.
- **Nothing here is a prediction of your outcome.** The plan improves your odds. It does not guarantee an offer, and three months is a short window in a competitive junior market.

> **If you do only three things from this report:** learn Power BI, deploy ImputeIQ to Azure with the benchmark published, and start sending ten applications a week from week three. Those three account for most of the achievable improvement in your position. Everything else is refinement.